

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:40

Annexure A

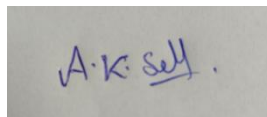
Industry Problem Solving Task

Code of Conduct:

A K Selva Pabhu (192421224) this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in blue ink that reads "A.K. Selva".

Annexure A

Industry Problem Solving Task

1. Smart Banking and Financial Management System

A financial institution develops a **Smart Banking and Financial Management System** to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The application follows a layered object-oriented architecture consisting of a presentation layer, service layer, domain layer, and data access layer connected to an OODBMS.

The system uses the **Factory Pattern** to create different account types, the **Strategy Pattern** for loan-interest and payment calculation policies, and the **Observer Pattern** for transaction and fraud alerts. A **Repository Pattern** is used to manage communication with persistent banking objects. The architecture is designed to support future services such as investment management and digital wallets.

Question:

Analyze how object-oriented principles, layered architecture, and design patterns contribute to the scalability, security, flexibility, and maintainability of the banking system. Explain how the application layers communicate with the OODBMS to store and retrieve persistent banking objects.

Answer:

The Smart Banking and Financial Management System manages customer accounts, fund transfers, loan processing, bill payments, and transaction history. The system uses a layered object-oriented architecture consisting of the Presentation Layer, Service Layer, Domain Layer, and Data Access Layer connected to an OODBMS.

Object-Oriented Principles

Encapsulation:

Customer account details, transaction information, and loan data are protected inside their respective classes. Access is provided through appropriate methods, improving security and data control.

Abstraction:

Complex operations such as fund transfer and loan processing are hidden behind simple service methods. Users do not need to know the internal implementation.

Inheritance:

Different account types such as Savings Account and Current Account can inherit common properties and methods from a common Account class.

Polymorphism:

Different account types can implement common operations such as calculating interest or processing transactions in their own way.

Layered Architecture**Presentation Layer:**

Provides the user interface for customers and bank employees to perform operations such as checking balances and transferring funds.

Service Layer:

Handles business operations such as fund transfers, loan processing, and bill payments.

Domain Layer:

Contains important banking objects and business rules such as Account, Customer, Loan, and Transaction.

Data Access Layer:

Communicates with the OODBMS and performs storage and retrieval of persistent banking objects.

Design Patterns**Factory Pattern:**

Creates different account objects such as Savings Account, Current Account, and other account types without exposing object creation logic.

Strategy Pattern:

Allows different loan-interest and payment calculation policies to be selected dynamically.

Observer Pattern:

Automatically sends transaction alerts and fraud alerts to registered users or monitoring systems whenever important events occur.

Repository Pattern:

Provides a simple interface for storing, retrieving, updating, and deleting persistent banking objects.

Communication with OODBMS

The Presentation Layer sends requests to the Service Layer. The Service Layer applies business rules and works with objects in the Domain Layer. When data needs to be stored or retrieved, the Domain Layer communicates through the Data Access Layer and Repository with the OODBMS.

The OODBMS stores objects such as Customer, Account, Loan, and Transaction in persistent form. When required, the Data Access Layer retrieves these objects and provides them to the Service Layer.

Benefits

The use of object-oriented principles and design patterns makes the system **scalable, secure, flexible, and maintainable**. New services such as investment management and digital wallets can be added without major changes to existing components. Layer separation also makes testing and maintenance easier.

2. Smart Retail and Inventory Management System

A retail organization develops a **Smart Retail and Inventory Management System** to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. The system follows a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system applies the **MVC Pattern** for managing user interfaces, the **Factory Pattern** for creating different product categories, and the **Observer Pattern** to notify managers when inventory reaches a minimum threshold. The **DAO Pattern** handles communication with persistent inventory objects. The architecture allows future integration of automated stock prediction and robotic warehouse management.

Question:

Evaluate how object-oriented design principles, layered architecture, and design patterns improve the flexibility, scalability, and maintainability of the retail system. Explain how the different layers interact with the OODBMS for efficient inventory persistence and retrieval.

Answer:

The Smart Retail and Inventory Management System manages products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. It follows a layered architecture consisting of Presentation, Business, Domain, and Data Access Layers connected to an OODBMS.

Object-Oriented Design Principles

Encapsulation:

Product, supplier, customer, and stock information are maintained inside appropriate classes and accessed through defined methods.

Abstraction:

Complex inventory operations are hidden behind simple business methods, making the system easier to use.

Inheritance:

Different product categories can inherit common features from a general Product class.

Polymorphism:

Different product categories can implement common operations according to their specific requirements.

Layered Architecture**Presentation Layer:**

Provides screens for searching products, managing stock, processing sales, and viewing reports.

Business Layer:

Processes business rules such as stock updates, purchase orders, and sales transactions.

Domain Layer:

Contains objects such as Product, Supplier, Customer, PurchaseOrder, and Inventory.

Data Access Layer:

Communicates with the OODBMS for storing and retrieving inventory objects.

Design Patterns**MVC Pattern:**

Separates the user interface into Model, View, and Controller components. This makes the user interface easier to manage and modify.

Factory Pattern:

Creates different product category objects without directly depending on their concrete classes.

Observer Pattern:

Notifies managers when inventory reaches the minimum stock threshold.

DAO Pattern:

Provides methods for storing, retrieving, updating, and deleting persistent inventory objects from the OODBMS.

Interaction with OODBMS

The Presentation Layer sends requests to the Business Layer. The Business Layer processes the required operation and interacts with Domain objects. The Data Access Layer uses DAO objects to communicate with the OODBMS.

The OODBMS stores persistent objects such as products, suppliers, customers, and inventory records. When stock information is required, the DAO retrieves the corresponding objects and sends them back to the upper layers.

Benefits

The architecture improves **flexibility, scalability, and maintainability**. New product categories can be added using the Factory Pattern. Inventory notifications can be extended using the Observer Pattern. Future technologies such as automated stock prediction and robotic

warehouse management can also be integrated without completely redesigning the existing system.

3. Smart Hotel and Hospitality Management System

A hotel chain develops a **Smart Hotel Management System** to manage guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. The application uses a layered object-oriented architecture consisting of presentation, service, domain, and data access layers connected to an OODBMS.

The system implements the **Strategy Pattern** for different pricing and discount policies, the **Factory Pattern** for creating room and service objects, and the **Observer Pattern** for reservation and service notifications. A **Repository Pattern** is used to simplify communication between business components and persistent hotel objects. Future enhancements include smart-room automation and personalized guest services.

Question:

Analyze how object-oriented principles and design patterns support flexibility, reusability, and maintainability in the hotel management system. Explain how the layered architecture coordinates reservation processing and communicates with the OODBMS for persistent data management.

Answer:

The Smart Hotel Management System manages guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. It uses Presentation, Service, Domain, and Data Access Layers connected to an OODBMS.

Object-Oriented Principles

Encapsulation:

Guest details, room information, reservation details, and payment information are protected within their respective classes.

Abstraction:

Complex operations such as room booking and payment processing are hidden behind simple methods.

Inheritance:

Different room types such as Standard, Deluxe, and Suite can inherit common properties from a Room class.

Polymorphism:

Different room and service types can implement common operations differently according to their requirements.

Layered Architecture

Presentation

Layer:

Provides the interface for guests and hotel employees to make reservations, check rooms, process payments, and manage services.

Service

Layer:

Handles business operations such as reservation processing, pricing, check-in/check-out, and payment.

Domain Layer:

Contains objects such as Guest, Room, Reservation, Payment, and RoomService.

Data Access Layer:

Communicates with the OODBMS to store and retrieve persistent hotel objects.

Design Patterns

Strategy Pattern:

Supports different pricing and discount policies. For example, different strategies can be applied for seasonal pricing, membership discounts, or special offers.

Factory Pattern:

Creates different room and service objects based on the required type.

Observer Pattern:

Sends notifications when reservations or room services are created or updated.

Repository Pattern:

Provides a simplified way for business components to access persistent hotel objects.

Reservation Processing

When a guest makes a reservation, the request first reaches the Presentation Layer. It is passed to the Service Layer, which validates availability and applies pricing rules. The Domain Layer represents the reservation and room objects. The Repository in the Data Access Layer stores the reservation in the OODBMS.

When reservation information is required later, the Repository retrieves the corresponding persistent objects and returns them to the Service Layer.

Benefits

The system becomes **flexible, reusable, and maintainable** because each layer has a separate responsibility. Design patterns reduce code duplication and make it easier to introduce future enhancements such as smart-room automation and personalized guest services.

4. Smart Logistics and Delivery Management System

A logistics company develops a **Smart Logistics and Delivery Management System** to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. The system adopts a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system uses the **Strategy Pattern** for selecting optimal delivery routes, the **Factory Pattern** for creating different shipment types, and the **Observer Pattern** for real-time delivery status notifications. A **DAO Pattern** manages persistent shipment and vehicle information. The architecture is designed to support future integration with autonomous delivery vehicles and AI-based route optimization.

Question:

Examine how object-oriented design principles, layered architecture, and design patterns improve the adaptability, scalability, and maintainability of the logistics system. Explain how the layers communicate with the OODBMS to manage persistent shipment, vehicle, and delivery information.

Answer:

The Smart Logistics and Delivery Management System manages shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. The system uses Presentation, Business, Domain, and Data Access Layers connected to an OODBMS.

Object-Oriented Principles

Encapsulation:

Shipment, vehicle, warehouse, and delivery information are maintained inside appropriate objects.

Abstraction:

Complex route selection and delivery operations are hidden behind simple methods.

Inheritance:

Different shipment types can inherit common properties from a general Shipment class.

Polymorphism:

Different shipment types can implement delivery-related operations according to their specific requirements.

Layered Architecture

Presentation Layer:

Provides interfaces for registering shipments, allocating vehicles, tracking deliveries, and viewing delivery status.

Business Layer:

Handles business operations such as vehicle allocation, shipment processing, and route selection.

Domain Layer:

Contains objects such as Shipment, Vehicle, Warehouse, Delivery, and Customer.

Data Access Layer:

Manages communication with the OODBMS for persistent shipment and vehicle information.

Design Patterns**Strategy Pattern:**

Selects the most suitable delivery route based on different route optimization strategies.

Factory Pattern:

Creates different shipment objects such as standard, express, or special shipment types.

Observer Pattern:

Provides real-time delivery status notifications to customers and other registered users.

DAO Pattern:

Handles storing, retrieving, updating, and deleting shipment and vehicle information in the OODBMS.

Communication with OODBMS

The Presentation Layer sends requests to the Business Layer. The Business Layer processes the request using Domain objects. The Data Access Layer uses DAO objects to communicate with the OODBMS.

The OODBMS stores persistent objects such as Shipment, Vehicle, Warehouse, and Delivery. When tracking information or vehicle details are required, the DAO retrieves the corresponding objects from the database.

Benefits

The architecture improves **adaptability, scalability, and maintainability**. New shipment types can be added using the Factory Pattern, while new route optimization methods can be introduced using the Strategy Pattern. Future technologies such as autonomous delivery vehicles and AI-based route optimization can be integrated more easily.

5. Smart Energy Management System

An energy management company develops a **Smart Energy Management System** to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for abnormal consumption. The system follows a layered object-oriented architecture where the

presentation layer provides dashboards, the business layer performs energy analysis, the domain layer manages energy-related rules, and the data access layer communicates with an OODBMS.

The system uses the **Observer Pattern** for consumption alerts, the **Strategy Pattern** for different energy optimization algorithms, and the **Factory Pattern** for creating different smart-meter objects. A **Repository Pattern** is used to manage persistent energy data. The architecture is designed to accommodate future renewable-energy monitoring and AI-based consumption prediction.

Question:

Evaluate how object-oriented principles, layered architecture, and design patterns contribute to the scalability, extensibility, and maintainability of the energy management system. Explain the interaction between the system layers and the OODBMS for storing, retrieving, and managing persistent energy-monitoring objects.

Answer:

The Smart Energy Management System monitors electricity consumption, manages smart meters, analyzes energy usage, and generates alerts for abnormal consumption. It uses Presentation, Business, Domain, and Data Access Layers connected to an OODBMS.

Object-Oriented Principles

Encapsulation:

Smart meter information, energy consumption data, and analysis results are maintained inside appropriate objects.

Abstraction:

Complex energy analysis and optimization operations are hidden behind simple methods.

Inheritance:

Different smart meter types can inherit common properties and operations from a general SmartMeter class.

Polymorphism:

Different smart meter types can perform monitoring operations according to their specific characteristics.

Layered Architecture

Presentation Layer:

Provides dashboards for displaying electricity consumption, energy usage reports, and alerts.

Business Layer:

Performs energy analysis and applies energy optimization operations.

Domain Layer:

Manages energy-related objects and rules such as SmartMeter, EnergyReading, Consumption, and Alert.

Data Access Layer:

Communicates with the OODBMS to store and retrieve persistent energy data.

Design Patterns**Observer Pattern:**

Generates alerts when abnormal electricity consumption is detected. Registered users or monitoring systems are notified automatically.

Strategy Pattern:

Supports different energy optimization algorithms. Different strategies can be selected based on the required optimization method.

Factory Pattern:

Creates different smart-meter objects according to the required meter type.

Repository Pattern:

Provides a simple interface for storing, retrieving, updating, and managing persistent energy objects.

Interaction with OODBMS

The Presentation Layer sends requests to the Business Layer. The Business Layer performs energy analysis using Domain objects. When energy data needs to be stored or retrieved, the Data Access Layer communicates with the OODBMS through the Repository.

The OODBMS stores objects such as SmartMeter, EnergyReading, ConsumptionData, and Alert. These objects can be retrieved whenever dashboards, reports, or analysis operations require them.

Benefits

The combination of object-oriented principles, layered architecture, and design patterns provides **scalability, extensibility, and maintainability**. New smart-meter types can be added using the Factory Pattern, new optimization algorithms can be introduced through the Strategy Pattern, and new alert mechanisms can be implemented using the Observer Pattern.

The architecture can also support future features such as **renewable-energy monitoring and AI-based consumption prediction** without major changes to the existing system.

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and	Logical and feasible	Basic solution with limited	Unclear or impractical

		feasible solution aligned with industry standards	solution with minor gaps	feasibility	solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation